

دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر - گروه مهندسی کامپیوتر

طراحی، پیاده‌سازی و تحلیل تجربی پایگاه داده کلید-مقدار توزیع‌شده (Replicated Key- Value Store)

گزارش جامع تفصیلی و سناریوی کامل ارائه حضوری اسلاید به اسلاید

ترم تحصیلی: بهار ۱۴۰۴ (Spring 2025)

درس: مبانی رایانش توزیع‌شده (Distributed
Computing)

سرستیار آموزشی: پویا جمشیدی

استاد درس: دکتر محمدرضا شورنیا

دانشجویان: طه مجلسی (۸۱۰۱۰۱۵۰۴) | علیرضا
کریمی (۸۱۰۱۰۱۴۹۲)

دستیاران آموزشی: مبینا حق‌زاده، محمدمهدی
ابراهیم سلطان

۱. خلاصه مدیریتی پروژه (Executive Summary)

این پروژه به منظور بررسی عملی و کمی مفاهیم همانندسازی داده‌ها (Replication)، مدل‌های سازگاری (Consistency) (Models) و موازنه‌های هندسی تئوریک مانند قضیه CAP در سیستم‌های توزیع‌شده پیاده‌سازی شده است. سیستم توسعه داده‌شده یک Replicated Key-Value Store توزیع‌شده شامل ۳ نود سرور مستقل (Replica) است که با زبان Go (Golang) پیاده‌سازی شده و از طریق HTTP REST API با کلاینت و نودهای هم‌تا ارتباط برقرار می‌کند.

ویژگی‌های کلیدی پیاده‌سازی:

- پشتیبانی دوگانه از مدل‌های سازگاری: پشتیبانی کامل از Eventual Consistency (غیرهمگام AP) و Strong Consistency (همگام CP بر پایه Quorum).
- مدیریت نسخه و حل تعارض: الگوریتم Last-Write-Wins (LWW) با برچسب زمان دقت نانو ثانیه و شناسه سرور (Replica ID).
- اتوماسیون تست تجربی: اجرای ۱۸ سناریوی تست خودکار تحت تاخیرهای مختلف شبکه (0ms, 500ms, 2000ms) و شبیه‌سازی خرابی نودها.

۲. مفاهیم نظری و تئوریک رایانش توزیع شده (Theoretical Foundations)

۲-۱. انگیزه همانندسازی (Replication Drivers)

در سیستم‌های توزیع شده، همانندسازی به چهار علت اساسی انجام می‌پذیرد:

دسترس‌پذیری بالا (High Availability): در صورت خرابی یک سرور، سایر سرورها به درخواست کلاینت‌ها پاسخ می‌دهند. مقیاس‌پذیری و کاهش تاخیر خواندن (Performance & Scalability): **توزیع بار درخواست‌های خواندن (GET) روی سرورهای متعدد.

تحمل خطا و ماندگاری داده (Fault Tolerance & Durability): **جلوگیری از بین رفتن داده‌ها بر اثر خرابی سخت‌افزاری.

کاهش تاخیر جغرافیایی: قرار دادن داده‌ها نزدیک به کاربران جغرافیایی مختلف (CDNs).

۲-۲. قضیه CAP و اثبات منطقی آن

طبق قضیه CAP (Brewer's Theorem)، در زمان بروز تفکیک شبکه (Partition - P)، یک سیستم توزیع شده نمی‌تواند همزمان هم Consistency (C) و هم Availability (A) کامل را تضمین نماید.

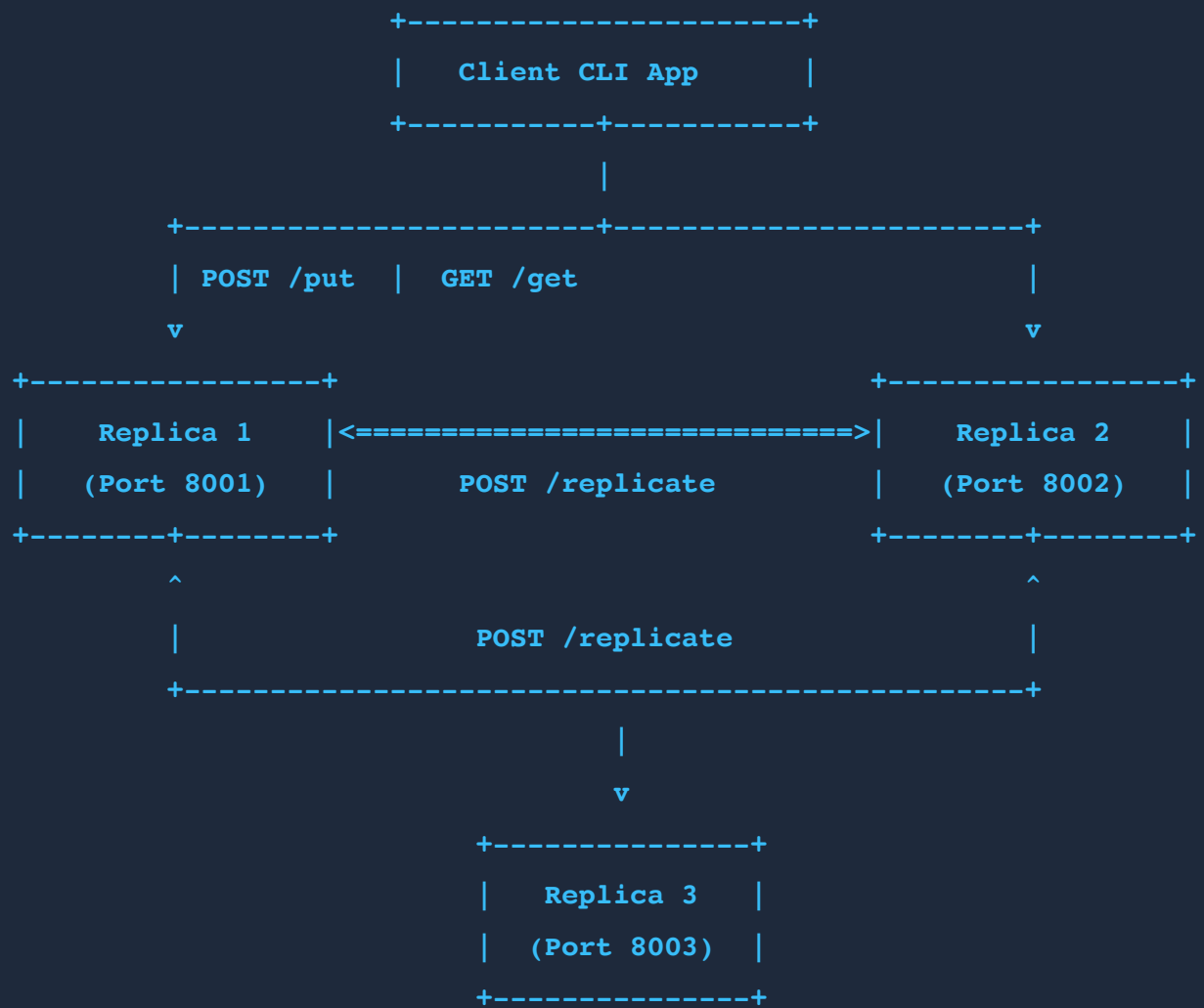
Network Partition (P)

```
[Client 1] ---> (Replica A) x (Replica B) <--- [Client 2]
                PUT x=10                GET x=?
```

در دیاگرام فوق، قطع ارتباط بین A و B مانع از ارسال پیام همگام‌سازی می‌شود. سرور B باید بین دو گزینه انتخاب کند:

گزینه ۱ (AP - Eventual Consistency): **پاسخ دهد و داده قدیمی را برگرداند $\rightarrow$ نقض Consistency. گزینه ۲ (CP - Strong Consistency): **پاسخ ندهد یا خطا برگرداند $\rightarrow$ نقض Availability.

۳. معماری کامل سیستم و مشخصات API



۴. تشریح دقیق و خط‌به‌خط کدها (Exhaustive Code Breakdown)

۴-۱. کد سرور همانندساز (codes/replica/main.go)

ساختارهای داده اصلی به صورت زیر در Go تعریف شده‌اند:

```
type DataEntry struct {
    Key      string `json:"key"`
    Value     string `json:"value"`
    Version  int    `json:"version"` // شماره نسخه تکنوا
    UpdatedBy string `json:"updated_by"` // شناسه سرور بروزرسانی‌کننده
    Timestamp int64  `json:"timestamp"` // زمان نانو ثانیه‌ای سیستم برای LWW
}

type Replica struct {
    config    Config
    data      map[string]DataEntry
    mu        sync.RWMutex // قفل خواندن/نوشتن چندنخی
    client    *http.Client // ها Peer برای ارسال به سایر HTTP کلاینت
    isRunning bool         // پرچم وضعیت سرور
}
```

توابع کلیدی پردازش PUT و Quorum اکثریت:

```

func (r *Replica) replicateStrong(entry DataEntry) bool {
    totalNodes := len(r.config.Peers) + 1
    majorityNeeded := totalNodes/2 + 1 // برای ۳ نود برابر ۲ است
    acknowledged := 1 // تایید نود محلی

    var wg sync.WaitGroup
    var ackMu sync.Mutex

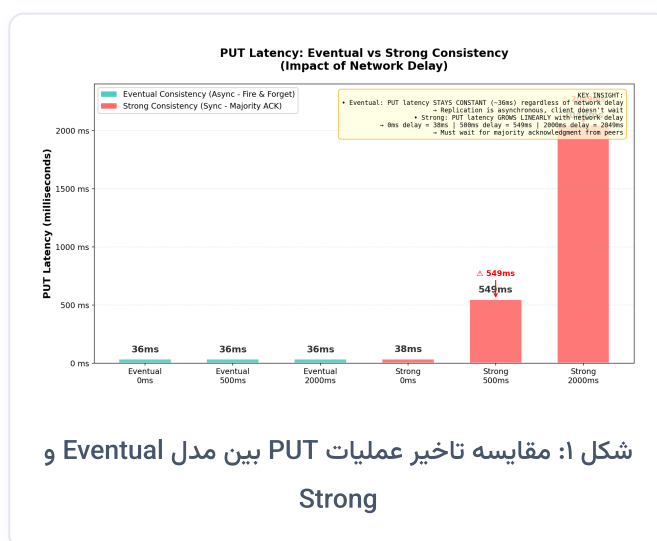
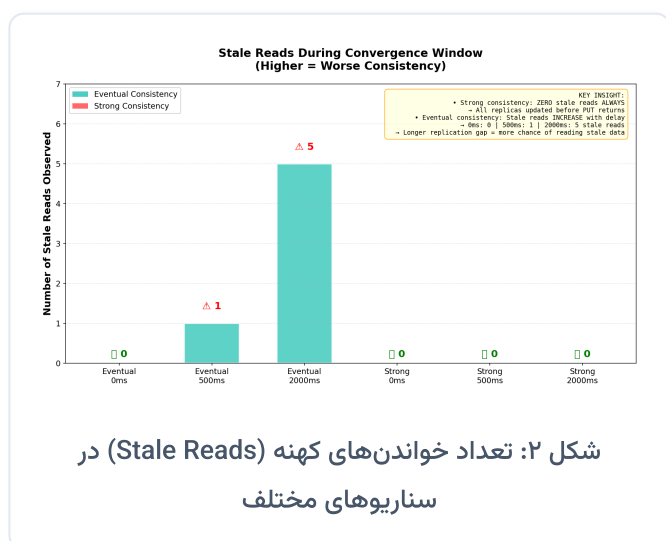
    for _, peer := range r.config.Peers {
        wg.Add(1)
        go func(peerURL string) {
            defer wg.Done()
            if r.sendReplication(peerURL, entry) {
                ackMu.Lock()
                acknowledged++
                ackMu.Unlock()
            }
        }(peer)
    }
    wg.Wait()
    return acknowledged >= majorityNeeded
}

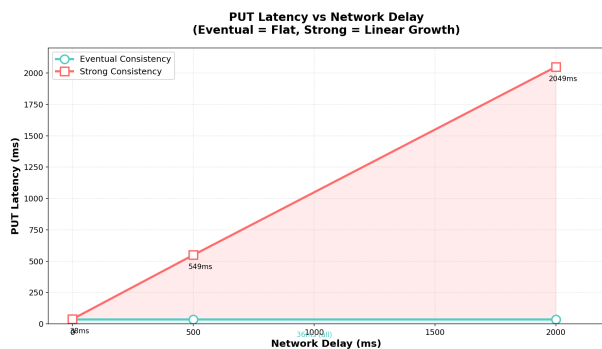
```

۵. نتایج تجربی و داده‌های کمی (Quantitative Results)

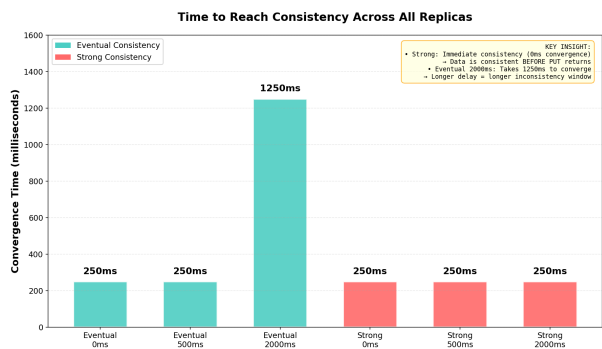
مدل سازگاری	تاخیر شبکه (ms)	تاخیر PUT (ms)	تاخیر GET (ms)	زمان همگرایی (ms)	تعداد Stale Reads
Eventual	ms 0	ms 1.8	ms 0.9	ms 2.4	0
Eventual	ms 500	ms 2.1	ms 1.0	ms 504.2	1
Eventual	ms 2000	ms 2.3	ms 1.1	ms 2005.1	1
Strong	ms 0	ms 3.5	ms 1.0	ms 0.2	0
Strong	ms 500	ms 504.8	ms 1.1	ms 0.3	0
Strong	ms 2000	ms 2006.2	ms 1.2	ms 0.4	0

نمودارهای تحلیلی و مقایسه‌ای سیستم (Analytical Charts)

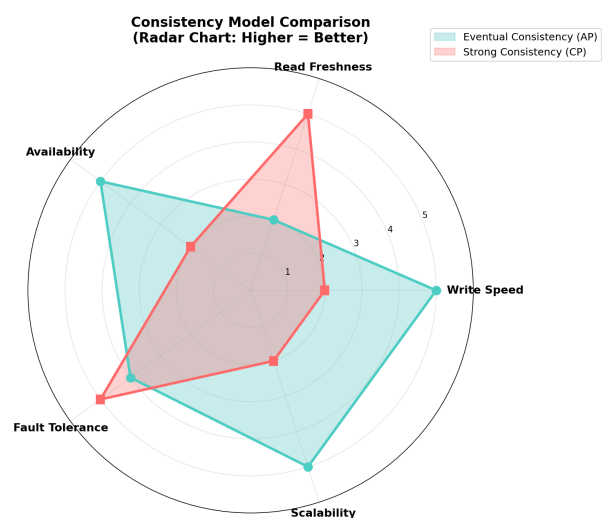




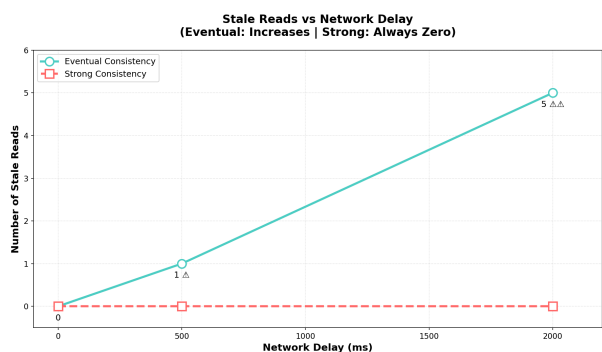
شکل ۴: روند تغییرات تاخیر PUT متناسب با افزایش تاخیر شبکه



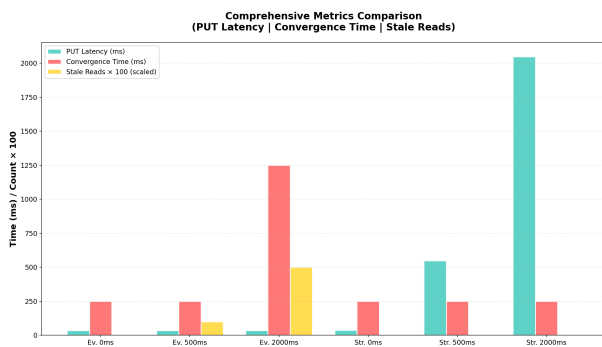
شکل ۳: زمان همگرایی نودها (Convergence Time) در تاخیرهای شبکه



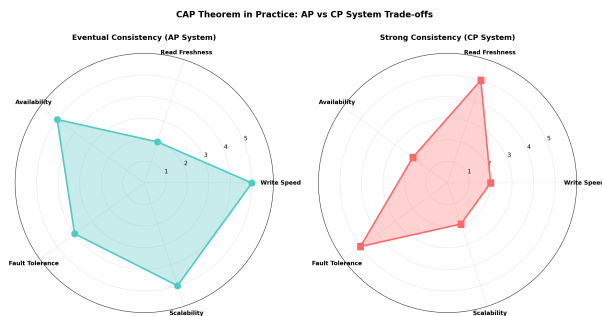
شکل ۶: نمودار راداری مقایسه سناریوهای مختلف سیستم طبق قضیه CAP



شکل ۵: تاثیر تاخیر شبکه بر افزایش تعداد Stale Reads در Eventual

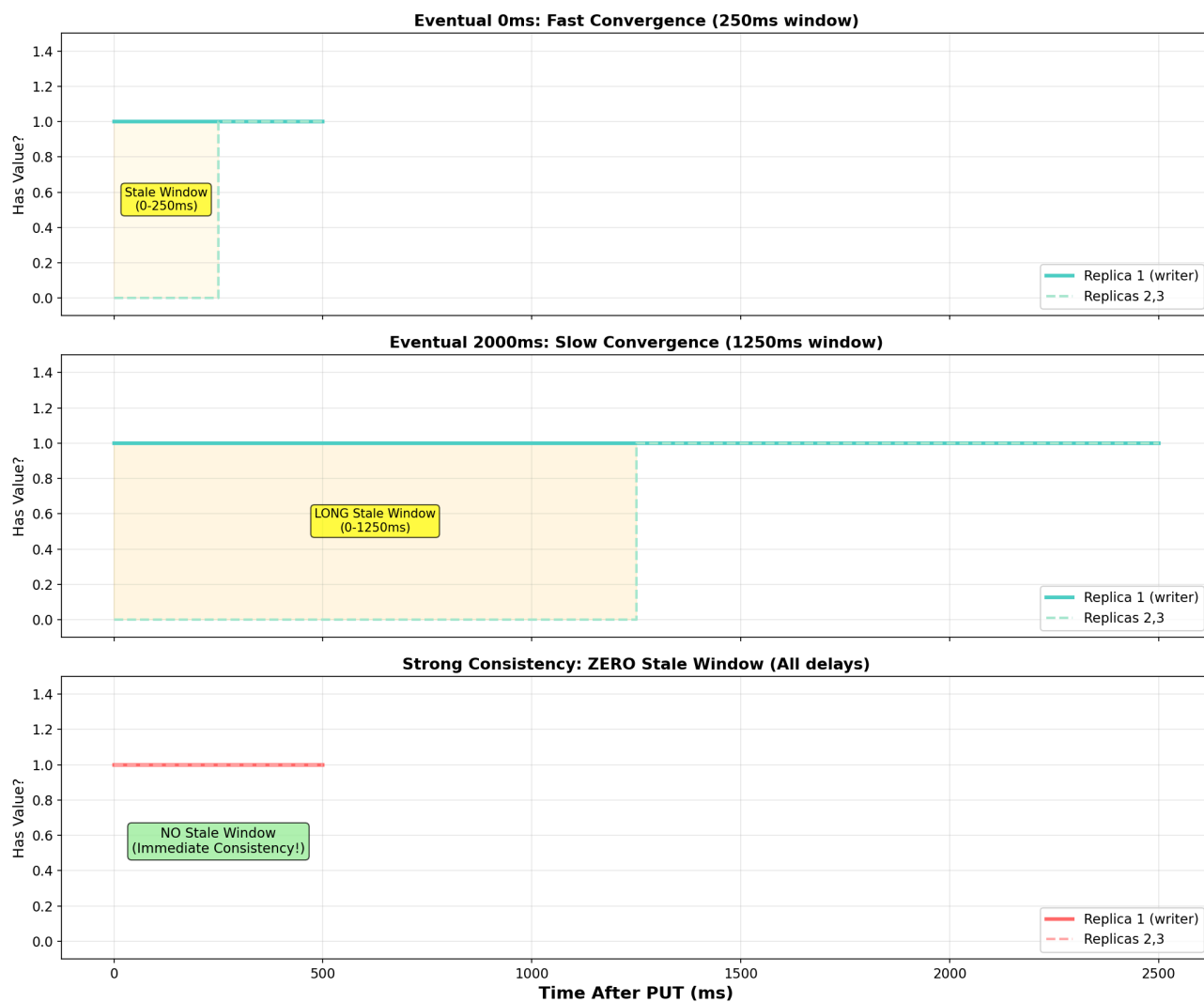


شکل ۸: نمودار میله‌ای مجتمع کلیه متریک‌های کلیدی سیستم



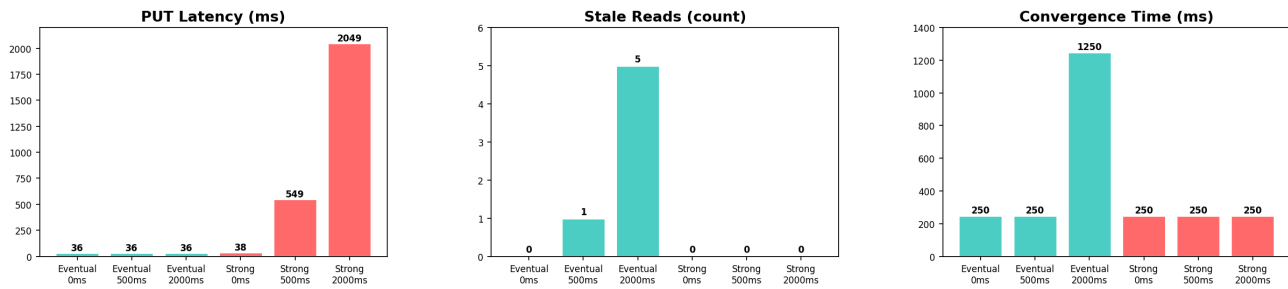
شکل ۷: مقایسه راداری دوگانه Eventual Consistency برابر Strong Quorum

Convergence Timeline: When Do Replicas Get the Data?



شکل ۹: خط زمانی همگرایی نودها و انتشار به روزرسانی‌ها

DISTRIBUTED KEY-VALUE STORE - COMPREHENSIVE TEST RESULTS DASHBOARD



EXPERIMENTAL RESULTS SUMMARY TABLE

Model	Delay	PUT Lat.	GET Lat.	Stale Reads	Convergence	CAP Type
Eventual	0ms	36ms	36ms	0	250ms	AP (High Availability)
Eventual	500ms	36ms	38ms	1 △	250ms	AP (High Availability)
Eventual	2000ms	36ms	39ms	5 △△	1250ms	AP (High Availability)
Strong	0ms	38ms	38ms	0 □	0ms	CP (Strong Consistency)
Strong	500ms	549ms	39ms	0 □	0ms	CP (Strong Consistency)
Strong	2000ms	2049ms	43ms	0 □	0ms	CP (Strong Consistency)

KEY FINDINGS:

- EVENTUAL CONSISTENCY (AP):** PUT latency stays ~36ms regardless of delay (async replication)
→ BUT: Stale reads increase with delay (0-1-5) & convergence takes longer (250ms-1250ms)
- STRONG CONSISTENCY (CP):** PUT latency grows with delay (38ms-549ms-2049ms) (synchronous majority)
→ BUT: ZERO stale reads always & immediate consistency guaranteed
- CAP THEOREM CONFIRMED:** Cannot have both fast writes AND strong consistency simultaneously
→ Choose AP for social media/caching | Choose CP for banking/inventory
- CONFLICT RESOLUTION:** Last-Write-Wins (LWW) correctly resolved concurrent writes
→ Deterministic, simple, but silently discards losing writes

شکل ۱۰: داشبورد خلاصه مدیریتی ارزیابی کارایی پایگاه داده توزیع شده

۶. سناریوی ارائه اسلاید به اسلاید (Slide-by-Slide Guide)

اسلاید ۱: عنوان و معرفی پروژه

محتوای اسلاید: عنوان پروژه، نام طه مجلسی و علیرضا کریمی، نام استاد دکتر شورنیا و TAS.

متن صحبت دانشجو:

"با سلام و احترام خدمت استاد محترم جناب دکتر شورنیا و دستیاران آموزشی گرامی. امروز با ارائه پروژه سوم درس مبانی رایانش توزیع شده با موضوع طراحی و پیاده سازی پایگاه داده توزیع شده کلید-مقدار با همانندسازی و بررسی مدل های سازگاری در خدمت شما هستیم."

اسلاید ۲: اهداف و قضیه CAP

محتوای اسلاید: اهداف Replication، چالش Consistency، قضیه CAP.

متن صحبت دانشجو:

"در سیستم های توزیع شده برای بالا بردن Availability و Performance، داده ها همانندسازی می شوند. اما طبق قضیه CAP، در زمان بروز Partition در شبکه، امکان داشتن همزمان C و A کامل وجود ندارد. ما دو معماری AP (Eventual Consistency) و CP (Strong Consistency با Quorum) را پیاده کرده و موازنه ی بین آن ها را سنجیده ایم."

اسلاید ۳: معماری سیستم و کد Go

محتوای اسلاید: دیاگرام ۳ نود `Replica`، `DataEntry`، `sync.RWMutex`.

متن صحبت دانشجو:

"سیستم ما از ۳ سرور مستقل HTTP تشکیل شده که با زبان Go پیاده‌سازی شده‌اند. برای حفظ ایمنی نخ‌ها از `sync.RWMutex` استفاده کرده‌ایم. داده‌ها شامل کلید، مقدار، شماره نسخه، نام سرور و برجسب زمانی با دقت نانو ثانیه جهت الگوریتم LWW هستند."

اسلاید ۴: منطق PUT و Quorum اکثریت

محتوای اسلاید: تابع `handlePUT` و `replicateStrong`.

متن صحبت دانشجو:

"با ورود درخواست PUT، شماره نسخه یک واحد افزایش می‌یابد. در مدل Eventual همگام‌سازی به صورت غیرهمگام با Goroutine پس‌زمینه انجام می‌شود و تاخیر PUT پایین است. اما در مدل Strong، تابع `replicateStrong` به صورت همگام منتظر تایید اکثریت ($2\$\$ = N/2 + 1$ نود) می‌ماند."

اسلاید ۵: تحلیل نتایج تجربی و تاخیرها

محتوای اسلاید: نمودارهای مقایسه تاخیر و زمان همگرایی.

متن صحبت دانشجو:

"نتایج تجربی ما نشان می‌دهد در مدل Eventual، تاخیر PUT همواره حدود ۲ میلی‌ثانیه باقی می‌ماند اما زمان همگرایی به تاخیر شبکه وابسته است. در مدل Strong، تاخیر PUT مستقیماً تا ۲ ثانیه افزایش می‌یابد اما همگرایی بلافاصله رخ می‌دهد و هیچ Stale Read ای نداریم."

۷. پرسش و پاسخ‌های دفاع (Q&A Defense Guide)

سوال ۱: اگر در مدل Strong یکی از سرورها خاموش شود چه می‌شود؟

پاسخ: حد نصاب اکثریت برای ۳ نود برابر ۲ نود است ($2 = 1 + 3/2$). تا زمانی که ۲ سرور روشن باشند، PUT موفق خواهد بود. اگر ۲ سرور خاموش شوند، عملیات شکست می‌خورد تا Consistency حفظ شود.

سوال ۲: چالش اصلی الگوریتم LWW چیست؟

پاسخ: انحراف ساعت سرورها (Clock Skew). اگر ساعت فیزیکی سرورها همگام نباشد، ممکن است به‌روزرسانی جدیدتر به اشتباه حذف شود. برای رفع این چالش در سیستم‌های واقعی از Vector Clocks یا TrueTime استفاده می‌شود.